Лекция 1. Строки: хеши, префикс-функция, Рабин-Карп, Кнут-Моррис-Пратт, z-функция.

1. Введение

Строка – это последовательность символов из конечного алфавита Σ. Длина строки s обозначается |s|. Индексация начинается с 0.

Основные понятия:

- **Подстрока** s[i:j] – символы с позиции i до j-1.
- **Префикс** – подстрока, начинающаяся с первого символа: s[0:k].
- **Суффикс** – подстрока, заканчивающаяся последним символом: s[k:|s|].

Задачи:

- Поиск подстроки в тексте.
- Сравнение строк.
- Поиск периодичностей, палиндромов и др.

2. Хеширование строк

2.1 Полиномиальный хеш

Идея: сопоставить строке s длины n число – значение многочлена в точке x по модулю p.

Определение:

$$hash(s) = (s[0] * x^{n-1} + s[1] * x^{n-2} + \ldots + s[n-1] * x^0) \bmod p,$$

где x – основание (случайное или фиксированное), p – большое простое число.

Пример:

s = "abc"
x = 31, p = 10^9+7
hash = ('a'*31^2 + 'b'*31^1 + 'c'*31^0) mod p

2.2 Быстрое вычисление хеша для подстрок

Если известен хеш для s[i:j], можно вычислить хеш для s[i+1:j+1] за O(1):

$$hash(s[i+1:j+1]) = ((hash(s[i:j]) - s[i] * x^{j-i-1}) * x + s[j]) \bmod p.$$

Предвычисляем степени x по модулю p и хеши всех префиксов:

$$pref_hash[0] = 0$$

$$pref_hash[i+1] = (pref_hash[i] * x + s[i]) \bmod p$$

Тогда хеш подстроки s[l:r]:

$$hash(s[l:r]) = (pref_hash[r] - pref_hash[l] * x^{r-l}) \bmod p$$

2.3 Вероятность коллизий

Если p достаточно велико ($\sim 10^9$), а x выбирается случайно, вероятность совпадения хешей для разных строк мала ($\sim \frac{1}{p}$). На практике часто используют двойное хеширование (два разных p) для надёжности.

Псевдокод предвычисления:

```
x = random(1, p-1)          # Выбираем случайное основание для полинома (1 ≤ x < p)
pows[0] = 1                 # x^0 = 1
for i from 1 to n:          # Предвычисляем все степени x по модулю p
   pows[i] = (pows[i-1] * x) mod p      # x^i = x^(i-1) * x
   pref_hash[i] = (pref_hash[i-1] * x + s[i-1]) mod p  # Хеш префикса длины i
```

Функция получения хеша подстроки:

```
function get_hash(l, r):     # l - начало подстроки, r - конец (полуинтервал [l, r))
   return (pref_hash[r] - pref_hash[l] * pows[r-l]) mod p
```

3. Префикс-функция

Определение: для строки s длины n префикс-функция π[i] — длина наибольшего собственного (не равного всей строке) префикса, который является суффиксом для s[0:i+1].

Пример:

s = "ababcab"

π = [0, 0, 1, 2, 0, 1, 2]

3.1 Алгоритм вычисления за O(n)

Идея: используем значение π для предыдущих позиций.

```
function prefix_function(s):
   n = length(s)            # Длина строки
   π = array[n] filled with 0  # Массив для значений префикс-функции
   for i from 1 to n-1:     # Для каждого символа (кроме первого)
      j = π[i-1]            # Берём значение префикс-функции для предыдущей позиции
                            # j - длина текущего кандидата на префикс-суффикс

      # Пытаемся расширить префикс: если символы не совпадают,
      # переходим к меньшему префиксу-суффиксу
      while j > 0 and s[i] != s[j]:
         j = π[j-1]       # Берём префикс-функцию от предыдущего префикса

      # Если символы совпали, увеличиваем длину префикса-суффикса
      if s[i] == s[j]:
         j += 1
```

```
    π[i] = j            # Сохраняем результат
  return π
```

3.2 Поиск подстроки с помощью префикс-функции

Для поиска подстроки t в тексте s построим строку t + '#' + s и вычислим префикс-функцию. Где π[i] = |t|, там найдено вхождение.

```
function kmp_search(s, t):
  concat = t + '#' + s     # Собираем строку: образец + разделитель + текст
                           # Разделитель '#' гарантирует, что префикс-функция
                           # не превысит длину образца при поиске в тексте
  π = prefix_function(concat)     # Считаем префикс-функцию для объединённой строки
  positions = []           # Список позиций, где найден образец

  # Проходим по части, соответствующей тексту s
  for i from |t|+1 to |concat|-1:
    # Если префикс-функция равна длине образца, нашли вхождение
    if π[i] == |t|:
      # Вычисляем позицию в исходном тексте s
      positions.append(i - 2*|t|)
  return positions
```

4. Алгоритм Рабина-Карпа

Идея: сравнивать хеши подстрок текста с хешом образца.

Псевдокод:

```
function rabin_karp(s, t):
  n = |s|, m = |t|        # Длины текста и образца
  if n < m: return []     # Если текст короче образца, вхождений нет

  p = large_prime         # Большое простое число для модуля
  x = random(1, p-1)      # Случайное основание

  h_t = hash(t)           # Хеш образца (вычисляется отдельной функцией)
  h_s = hash(s[0:m])      # Хеш первого окна длины m в тексте

  positions = []          # Список позиций вхождений

  # Проверяем первое окно
  if h_s == h_t and s[0:m] == t:
    positions.append(0)  # Добавляем позицию 0 при совпадении
```

```
    # Проходим окном по тексту
    for i from 1 to n-m:
        # Пересчитываем хеш для нового окна через хеш предыдущего
        h_s = ( (h_s - s[i-1]*x^(m-1)) * x + s[i+m-1] ) mod p

        # Если хеши совпали, проверяем символы (чтобы избежать коллизий)
        if h_s == h_t and s[i:i+m] == t:
            positions.append(i)  # Добавляем позицию
    return positions
```

Сложность: в среднем O(n+m), в худшем O(n*m) (при частых коллизиях).

5. Алгоритм Кнута-Морриса-Пратта (KMP)

Алгоритм KMP использует префикс-функцию для поиска подстрок без возврата по тексту.

Псевдокод:

```
function kmp(s, t):
    π = prefix_function(t)   # Считаем префикс-функцию только для образца
    j = 0                    # Текущая позиция в образце
    positions = []           # Список позиций вхождений

    for i from 0 to |s|-1:   # Проходим по всем символам текста
        # Пока есть несовпадение, откатываемся по префикс-функции
        while j > 0 and s[i] != t[j]:
            j = π[j-1]       # Используем префикс-функцию для отката

        # Если символы совпали, двигаемся по образцу
        if s[i] == t[j]:
            j += 1

        # Если дошли до конца образца, нашли вхождение
        if j == |t|:
            positions.append(i - |t| + 1)  # Начальная позиция вхождения
            j = π[j-1]               # Продолжаем поиск (откатываемся для поиска перекрывающихся вхождений)
    return positions
```

Сложность: O(n + m).

6. Z-функция

Определение: для строки s длины n z[i] – длина наибольшего префикса s, совпадающего с префиксом подстроки s[i:].

Пример:

s = "abacaba"

z = [0, 0, 1, 0, 3, 0, 1]

6.1 Алгоритм вычисления за O(n)

Используем «отрезок совпадения» [l, r].

```
function z_function(s):
   n = |s|                # Длина строки
   z = array[n] filled with 0  # Массив z-значений
   l = 0, r = 0           # Границы самого правого отрезка совпадения [l, r]

   for i from 1 to n-1:     # z[0] не определяется или равно 0
      # Если i внутри текущего отрезка [l, r]
      if i <= r:
         # Используем ранее вычисленные значения для ускорения
         z[i] = min(r - i + 1, z[i - l])

      # Пытаемся расширить z[i] наивным сравнением
      while i + z[i] < n and s[z[i]] == s[i + z[i]]:
         z[i] += 1

      # Если расширили за пределы текущего отрезка, обновляем [l, r]
      if i + z[i] - 1 > r:
         l = i
         r = i + z[i] - 1
   return z
```

6.2 Поиск подстроки с помощью Z-функции

Аналогично KMP: строим строку t + '#' + s, вычисляем z-функцию. Где z[i] = |t|, там найдено вхождение.

```
function z_search(s, t):
   concat = t + '#' + s     # Аналогично KMP: образец + разделитель + текст
   z = z_function(concat)   # Вычисляем z-функцию для объединённой строки
   positions = []           # Список позиций вхождений

   # Проходим по части, соответствующей тексту s
   for i from |t|+1 to |concat|-1:
      # Если z[i] равно длине образца, нашли вхождение
      if z[i] == |t|:
         # Вычисляем позицию в исходном тексте
         positions.append(i - |t| - 1)
   return positions
```

7. Сравнение методов

Метод	**Время (препроцессинг + поиск)**	**Память**	**Особенности**
Наивный поиск	$O(n \cdot m)$	$O(1)$	Простой, медленный
Рабин-Карп	$O(m) + O(n)$ в среднем	$O(1)$	Вероятностный, быстро на практике
Префикс-функция	$O(m) + O(n)$	$O(m)$	Детерминированный, надёжный
KMP	$O(m) + O(n)$	$O(m)$	Без возвратов по тексту
Z-функция	$O(m) + O(n)$	$O(m + n)$	Удобен для своих задач